Ders 7 Genişletilmiş Notu: Yazılım Ölçümü, Test Planlama ve Vize Öncesi Genel Tekrar

Ders 7 Genişletilmiş Notu: Yazılım Ölçümü, Test Planlama ve Vize Öncesi Genel Tekrar

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Planı
- 2. Yazılım Ölçümünün Tekrarı: Halstead, McCabe, Büyüklük ve Güvenirlik
- 3. Ölçme, Ölçüt ve Ölçüm
- 4. Neden Ölçüyoruz?
- 5. Kalite Ölçütlerine Farklı Bakışlar: ISO 9126, McCall ve McConnell
- 6. Ölçme Süreci
- 7. CK Metrikleri
- 8. Yazılım Test Planlama ve Yönetimi
- 9. Test Planı ve Test Senaryosu
- 10. Test Ekibi ve Optimum Test Miktarı
- 11. Test Faaliyetlerinin Geliştirme Aşamalarına Dağılımı
- 12. Bakım Aşaması ve Yazılım Konfigürasyonu
- 13. Genel Tekrar: Yazılım Mühendisliğinde Kalite ve McCall Modeli
- 14. Kaliteli Yazılım İçin Aktiviteler ve Kalite Maliyeti
- 15. Standartlar ve Kalite Yönetim Sistemi
- 16. V Model ve Doğrulama-Geçerleme Düzeltmesi
- 17. Gözden Geçirme ve İnceleme
- 18. Test Süreci Adımları
- 19. Test Tipleri: Saydam Kutu, Kara Kutu, Gri Kutu
- 20. Hata Giderme ve Hata Düzeyleri
- Örnek Sorular

1. Dersin Planı

Hoca derse, bu haftanın üç bölümden oluşacağını söyleyerek başladı. Önce yazılım kalite ölçüm sistemleri konusuna, yazılım mühendisliği dersindeki notlardan hazırlanmış farklı bir bakış açısıyla eklemeler yapılacaktı. Sonra geçen hafta işlenen ve projelerde de kullanılacak olan test planı ve test spesifikasyon raporu konusu yeniden ele alınacaktı. Son olarak, bir sonraki hafta ara sınav olduğu için dönem başından beri görülen konuların genel tekrarı yapılacaktı. Öğrenciler genel tekrarın gerekli olduğunu söyleyince dersin ikinci yarısı tekrara ayrıldı.

Hoca tekrar slaytlarının özetlenmiş hâlini ders sonrasında öğrencilerin erişimine açacağını söyledi. Ancak tekrar slaytlarının yalnız temel noktaları içerdiğini, ayrıntılı ders slaytlarının da gözden kaçırılmaması gerektiğini vurguladı.

> **Hocanın vurgusu:** Konuları yalnız genel kavramlar olarak ezberlemeyin. Kavramları birbiriyle kıyaslayarak ve bu konularla ilgili sorulabilecek mantık sorularına cevap verebilecek şekilde çalışın. Derste görülenlerle ilgili yorum soruları, kavram kıyaslamaları ve belki küçük problem çözümleri olabilir.

Hoca ayrıca kalite iyileştirme konusunun bu tekrarın dışında kaldığını, sınavdan sonraki hafta işleneceğini belirtti.

2. Yazılım Ölçümünün Tekrarı: Halstead, McCabe, Büyüklük ve Güvenirlik

Önceki iki haftada yazılım kalite ölçümü ve değerlendirmesi işlenmişti: kalite ölçütlerinin hangi değerleri ölçtüğü, nerede kullanıldığı ve kimin için ne ifade ettiği (yöneticiler, yazılım mühendisleri gibi). Hoca bu kullanıcı gruplarının ayrıntısına bu kez girmedi, çünkü bu konu iki kez işlenmişti.

Yazılım mühendisliğinde yazılım ölçümü (software metrics) kavramı, yazılımın farklı kriterlere göre değerlendirilmesini sağlar: büyüklük açısından, karmaşıklık açısından ve iç yapı özellikleri açısından. Örneğin programdaki döngü sayıları ve döngülerin özellikleri sayılabilir. Yazılımın güvenirliği hakkında da yorum yapılabilir. Bunların hepsi, yazılımın nitelikleriyle ilişkili sayısal değerler alabildiğimiz maddeler, yani metriklerdir.

2.1 Halstead Metriği ve Algoritma Karmaşıklığı

Yazılım ölçümünde aritmetiksel değerler açısından da ölçme yapılabilir. Halstead metriğinde programın hacmi, programın uzunluğu, programın "gücü" ve aritmetiksel ifadelerin yoğunluğu gibi ölçme birimleri vardır. Bunun yanında, algoritmalar dersinden bilinen algoritma karmaşıklığı da yazılım hakkında sayısal bir değer verir.

> **Ek bilgi (derste anlatılmadı):** Halstead ölçüleri, programdaki farklı operatör sayısı n_1, farklı operand sayısı n_2, toplam operatör sayısı N_1 ve toplam operand sayısı N_2 üzerinden tanımlanır. Program uzunluğu $N = N_1 + N_2$, sözcük dağarcığı $n = n_1 + n_2$ ve program hacmi $V = N \log_2 n$ olarak hesaplanır. Zorluk ve çaba (effort) gibi diğer ölçüler de bu dört temel sayıdan türetilir.

2.2 McCabe Döngüsel Karmaşıklığı

McCabe, yazılımdaki döngüsel karmaşıklığı (cyclomatic complexity) tanımlamıştır. Bu değer, kontrol deyimlerinin yoğunluğundan kaynaklanan yolların, yani temel yolların sayısıyla ilişkilidir. Hesaplamada graf kullanılır:

- Düğümler program deyimlerini gösterir. McCabe ölçüsü için asıl önem kazanan düğümler kontrol deyimlerinin bulunduğu yerlerdir: bir `if` deyimi ya da bir döngü kontrolü.
- Kenarlar, düğümleri birbirine bağlayan yollardır.
- Döngüsel karmaşıklık değeri, kenar ve düğüm sayısından ya da graf çizildikten sonra oluşan kapalı alanların sayısından bulunur.

McCabe ölçüsü modül hakkında yargılara varmayı sağlar. Çok sayıda programlama projesi üzerinde yapılan gözlemlere dayanarak örneğin şu söylenebilir: Döngüsel karmaşıklık değeri 10'dan büyükse modülün bakım yapılabilirliği azalır.

> **Ek bilgi (derste anlatılmadı):** McCabe ölçüsü için yaygın formül $V(G) = E - N + 2$'dir. Burada E kenar sayısı, N düğüm sayısıdır. Düzlemsel bir akış grafında aynı değer, kapalı alan sayısına 1 eklenerek ya da karar (koşul) düğümü sayısına 1 eklenerek de bulunur.

2.3 Büyüklük Ölçüleri

Yazılımın büyüklüğü için function point ve satır sayısı kullanılır. Satır sayısı (LOC, line of code) ve bin satır (KLOC) büyüklük ölçüsüdür. Bu ölçülerle "modüller içinde şu kadar kod satırını geçenleri alt modüllere ayırmak iyi olur" gibi yargılara varılabilir.

Büyüklük satır sayısıyla ölçüldüğünde değer programlama diline bağlı olur. Bu yüzden diller arasında çevrim yapılır: bir dildeki belli sayıda satırın C dilinde kaç satıra karşılık geldiği gibi.

2.4 Güvenirlik

Güvenirlik de bir kalite özelliğidir. Güvenirlik modelleri istatistiksel sayımlarla kurulur: belli bir kullanım süresinde kaç hata çıktığı, bunlardan kaçının düzeltilebildiği ve toplam hata sayısı. Buna göre "sistem şu güvenirlik oranıyla çalışıyor" denebilir. Sistemde bir düzeltme yapılmışsa düzeltmeden sonra ne kadar hata kaldığı da değerlendirilir.

3. Ölçme, Ölçüt ve Ölçüm

Hoca, yazılım mühendisliği dersindeki yazılım ölçütleri konusundan üç kavramı tanımladı.

Ölçme (measuring): Somut veya soyut bir varlığın sahip olduğu bir özelliği sayısal ya da derecelendirilmiş bir veri olarak ifade etmektir. Hocanın örneği: "Ara sınav çok zordu" demek, çok genel de olsa derecelendirilmiş bir ölçü verir. Yazılımda ise asıl hedef, ölçmeyi sayısal değerlere ya da sayısal aralıklara getirmektir. Yazılımı doğru değerlendirecek özellikleri bulmak ve bunları daha doğru ölçebilmek amaçlanır.

Ölçüt (metric): Varlığın ölçülecek özelliğinin ölçme şeklidir. Sıcaklığı santigrat ya da Fahrenheit ile ölçmek buna örnektir. Yazılımda ölçüt örnekleri:

- algoritma karmaşıklığı,
- döngüsel karmaşıklık (cyclomatic complexity),
- CK metrik setindeki değerler: LCOM, bağlaşım (coupling), kohezyon, sınıf ağacının derinliği, ağırlıklı metot sayısı,
- büyüklük için LOC, KLOC ve function point.

Ölçüm (measurement): Belli bir ölçüte göre yapılan ölçme eyleminin sonucudur. Örneğin bir yazılım için function point ölçütü kullanılacaksa önce girdi sayısı, çıktı sayısı ve dosya sayısı sayılır. Bunlar doğrudan yapılan ölçümlerdir. Function point bu ölçümlerden hesaplanır, yani dolaylı bir ölçümdür.

> **Ek bilgi (derste anlatılmadı):** Klasik function point hesabında girdi, çıktı ve dosya sayılarının yanında kullanıcı sorgusu sayısı ve dış arayüz sayısı da sayılır. Bu beş değer karmaşıklık ağırlıklarıyla çarpılıp toplanır, sonra bir düzeltme katsayısıyla düzeltilir.

4. Neden Ölçüyoruz?

Ölçmenin amacı gerçek dünyayla ilgili, işe yarayacak anlamlı sonuçlar elde etmektir. Kalite açısından önemli değerleri iyileştirmek istiyorsak yazılımın neresinde ne değişiklik yapmamız gerektiğini bilmemiz gerekir. Bunu bilmek için de önce var olan durumu ölçmeliyiz. Yazılım karmaşık bir üründür. Ölçütler sayısal ve nicel iken yazılımın birçok özelliği tanımsaldır. Bu tanımsal durumlar nicel hâle dönüştürülmek istenir ve ölçüm sonuçları yorumlanır.

Hoca yazılımı neden ölçtüğümüzü şöyle sıraladı:

- Ne derecede iyi bir ürün ortaya çıkardığımızı bulmak için.
- Bu büyüklükte bir yazılımı geliştirirken ne kadar zaman ve para harcadığımızı bilmek için.
- Yeni başlayacağımız bir yazılımda büyüklük tahmini yapıp "bu büyüklükteki bir yazılım şu kadar maliyetle geliştirilebilir" diye yargıya varmak için.

  Hocanın vurgusu: "Ölçülemeyen yönetilemez." Bu, Tom DeMarco'nun klişe sözüdür.

5. Kalite Ölçütlerine Farklı Bakışlar: ISO 9126, McCall ve McConnell

Nitel kalite ölçütleri farklı kişilerce farklı biçimlerde düzenlenmiştir:

- **ISO 9126** kalite ölçütlerini tek tek sayar: işlevsellik, uygunluk, doğruluk, güvenlik, güvenirlik, olgunluk gibi.
- **McCall** aynı ölçütleri gruplara ayırarak sayar. Hoca bu grupları "işlevsel ölçütler, değiştirilebilme ölçütleri, çalıştırılabilme ölçütleri" diye andı.
- **McConnell** kalite ölçütlerini iç kalite ölçütleri ve dış kalite ölçütleri diye ikiye ayırır.

Yani aynı özellikler farklı metrik düzenlerinde farklı biçimde işlenebilir.

5.1 Dış Kalite Ölçütleri

Hoca bu ayrımı test türleriyle bağladı: Dış testte yazılımı dışarıdan bakan bir kişinin test etmesini anlıyorsak, McConnell'ın dış kalite ölçütleri de dışarıdan bakan kişinin yazılımdan isteyeceği özelliklerdir. Dışarıdan bakan kişi kullanıcıdır. Kullanıcı şu özellikler üzerinde durur:

- kullanışlılık,
- performans: hızlı cevap süresi, daha fazla veri tutma kapasitesi,
- yazılımı yetkisiz kişilerin kullanmasını önleyen güvenlik,
- doğru çalışma, yani istediği sonuçları doğru vermesi,
- bu doğruluğun güvenilir olması: güvenirlik (reliability).

5.2 İç Kalite Ölçütleri

İç kalite ölçütleri, yazılım geliştiricilerin program koduna veya sözde koda (pseudo code) bakarak ölçeceği değerlerdir. Yazılımı geliştirenleri ve bakımını yapacak kişileri ilgilendirirler:

- **Bakım kolaylığı (maintainability):** Yazılıma yetenek eklemenin, hatalarını gidermenin ya da başarımını artırmanın mümkün olduğunca kolay olması. Standart program yazma kurallarına göre yazılmış olmak bunu sağlar.
- **Esneklik (flexibility):** Yazılımın orijinal olarak tasarlandığı uygulamanın dışında çalışabilmesi için gereken değişikliklerin olabildiğince az olması.
- **Taşınabilirlik (portability):** Yazılımın farklı donanım ve işletim sistemleri gibi değişik çalışma ortamlarına kolayca aktarılabilmesi. Yazılımın farklı işletim sistemlerinde çalışabilecek şekilde genelleştirilmiş olması buna yardım eder.
- **Yeniden kullanılabilirlik (reusability):** Sistemin bütününün ya da parçalarının başka sistemlerde kolayca kullanılabilmesi.
- **Okunabilirlik (readability):** Kaynak kodun incelenmesinin kolay olması. Kodun, yazıldığı programlama dilinin standartlarına uygun yazılması, açıklamaların uygun yerlerde verilmesi ve dil kurallarına uygun bir tarz okunabilirliği artırır.
- **Anlaşılabilirlik (understandability):** Yazılımın sistem, bileşen ve kod düzeylerinde mümkün olduğunca kolay anlaşılması. Okunabilirlik yalnız kod düzeyindeki anlaşılabilirliği sağlar; anlaşılabilirlik daha geniştir.
- **Test edilebilirlik (testability):** Sistemin istenen gereksinimleri karşılayıp karşılamadığının bileşen ve tüm sistem düzeyinde test edilebilmesi.

6. Ölçme Süreci

Yazılım ölçümünde ölçme eylemi beş adımda gerçekleştirilir.

1. Tanımlama: Ölçütler, ölçülecek yazılıma uygun şekilde tanımlanır. Elimizdeki yazılımı en iyi hangi ölçütlerle tanımlayabileceğimize, yani hangi ölçme sistemlerini kullanacağımıza karar veririz. Burada iki şey önemlidir: kullanılan yaklaşım (örneğin yapısal programlama ya da başka bir yaklaşım) ve yazılımın türü (gerçek zamanlı mı, gömülü mü).

2. Toplama: Tanımlama aşamasında belirlenen ölçütlerin gerektirdiği veri elde edilir. Örneğin elimizde nesneye yönelik bir program varsa ve CK metriklerini seçtiysek, CK metrik setindeki maddelerin her biri için veri toplanır. Döngüsel karmaşıklık önemliyse graf çizilir ve temel yol sayısı bulunur. Büyüklük önemliyse satırlar sayılır ve KLOC değeri bulunur.

3. Hesaplama (analiz): Ham ölçülerden ölçütler hesaplanır. Matematiksel araçlar kullanılabilir. Hesaplamanın otomatik yapılması tercih edilir; bunun için altyapıyı otomatikleştirecek yazılımlar hedeflenir. Örnek: Veri toplama aşamasında girdi, çıktı ve dosya sayıları sayıldıysa, analiz aşamasında bunlardan function point bulunur.

4. Yorumlama (interpretation): Elde edilen ölçüm değerlerinden yararlı anlamlar çıkarılır. Örnek: "Function point değeri şu kadar çıktıysa bu yazılım büyük bir yazılımdır."

5. Geri besleme: Çıkarılan sonuçlar yazılım ekibine bildirilir ve ekip bu sonuçları kullanarak yazılımı iyileştirir. Örnek: "Şu kadar function point çıktı, bu büyük yazılım sınıfına girer." Aynı

şekilde karmaşıklık değeri ya da Halstead metriğiyle bulunan aritmetiksel karmaşıklık değeri de ekibe bildirilebilir.

Bir yazılımı iyileştirmek istiyorsak önce önceki değerlerini bilmemiz gerekir. Bu değerleri standart düzeylere çıkarmak için ne yapılması gerektiği yazılım geliştirenlere söylenir. Hoca, ölçme işini yapanların görevinin de bu adımlarla belirlendiğini söyledi.

7. CK Metrikleri

CK metrikleri nesneye yönelik yazılımlar için tanımlanmış ölçüm sistemleridir. Sınıflar üzerinden sayılırlar ve yalnız nesneye yönelik yaklaşımla geliştirilmiş yazılımlarda anlamlıdırlar. Metrikler şunlardır:

- sınıftaki ağırlıklı metot sayısı,
- kalıtım ağacının derinliği,
- alt sınıf sayısı,
- sınıfın yanıt kümesinin eleman sayısı,
- sınıflar arası bağlaşım,
- metotlardaki kohezyon (uyum) eksikliği.

  Ek bilgi (derste anlatılmadı): Bu metriklerin İngilizce kısaltmaları sırasıyla WMC (Weighted Methods per Class), DIT (Depth of Inheritance Tree), NOC (Number of Children), RFC (Response For a Class), CBO (Coupling Between Object classes) ve LCOM (Lack of Cohesion in Methods) şeklindedir. Set, Chidamber ve Kemerer'in adlarının baş harfleriyle anılır.

Bu ölçütlere eleştiriler de vardır: Her biri istenen değerin karşılığını tam veremez. Örneğin ağırlıklı metot sayısında metot karmaşıklığının neye göre belirleneceği, yani ölçütün nasıl rehberlik edeceği açık değildir. Çok sayıda metodu olan bir sınıf için "sorumlulukların dağıtılması zor olur" gibi değerlendirmeler yapılabilir.

Hoca bölümü şöyle bağladı: Ölçme sistemlerinde ölçme işlemini metriklerle, yani ölçütlerle yaparız. Ölçüt, hangi özelliği nasıl ölçeceğimizi belirler. Belli bir ölçüte göre yapılan ölçme eyleminin sonucuna da ölçüm (measurement) denir.

8. Yazılım Test Planlama ve Yönetimi

Aradan sonra hoca geçen hafta işlenen yazılım test planlama ve yönetimi konusunu tekrar etti. Bu bölümün hedefi şu sorulara cevap vermektir: Test spesifikasyonu belgesi neleri içerir? Test planında neler olur? Test senaryoları nasıl hazırlanır? Test işini kim yapar? Testte hata arama ne zamana kadar sürer, test nerede sonlandırılır?

8.1 Test Kapsamındaki İşlemler

Test kapsamında sırasıyla şu işlemler yapılır:

- **Test metodolojisine karar verme:** Aslında yazılım geliştirme modeline ve metodolojisine karar verildiğinde test metodolojisi de bunun içinde belirlenmiş olur.
- **Test planlama.**
- **Test tasarımı:** Test verisinin hazırlanmasına da bu sırada geçilir.
- **Testin gerçekleştirilmesi.**

8.2 Test Spesifikasyonu

Yazılım testlerinin tanımlanması, planlanması, düzenlenmesi ve belgelenmesi için bir test spesifikasyonu oluşturulur. Test planında hangi testlerin yapılacağı ve bunların zamanlaması verilir. Test adımları için, diğer derslerden bilinen bir Gantt diyagramı hazırlanır. Test için gider tahmini yapılır ve hangi test ortamlarının ve kaynaklarının kullanılacağı belirlenir. Test senaryolarında ise kullanılacak veri ve izlenecek senaryolar yer alır.

Spesifikasyonda her test için şu bilgiler oluşturulur:

- testin tanımı ve amacı,

- test edilen modüller,
- kullanılan özel araçlar (tool),
- testin gideri, yani ne kadar süre gerektiği,
- test programında kullanılacak veri,
- beklenen sonuçlar,
- gerçek sonuçlar.

Hoca bunu projeyle somutlaştırdı: Örneğin bir birim testi, bir sistem testi ve bir kullanıcı testi yapacağınızı planladınız. Bunların zaman planı ne olacak, ne kadar süre gerekecek? Gider derken kastedilen budur. Ortam ve kaynak olarak birim testinde, kullanılan programlama diline uygun bir birim test aracı ve ortamı seçilir. Sistem testi için de uygun özel araçlar seçilir.

8.3 Test Yönetiminin Geri Beslemeli Yapısı

Test yönetimi süreci geri beslemeli bir süreçtir. Hata mesajları sınıflandırılır ve hata mesajlarının içeriği belge olarak kaydedilir. IEEE standartlarından biri de test dokümantasyonunun, yani test spesifikasyonunun nasıl oluşturulacağını verir.

Test spesifikasyonu oluşturulur, incelenir, düzeltilir ve saklanır. Böylece bir baseline oluşturulur. Gereksinim analizi belgesi, tasarım belgesi ve kod nasıl yazılım konfigürasyon yönetiminde bir baseline'a bağlanıp her değişiklikte konfigürasyon belgesine kaydediliyorsa, test spesifikasyonu da yazılım geliştirme belgelerinden biri olarak aynı şekilde konfigürasyon yönetimine girer.

9. Test Planı ve Test Senaryosu

9.1 Test Planının İçeriği

Test planında şunlar bulunur:

- test stratejisi,
- test edilecek öğeler,
- doğrulama yöntemleri,
- testlerin tamamlanma kriterleri, yani hata aramaya ne zamana kadar devam edileceği,
- hata ve test sonuç raporunun biçimi,
- test sorumlulukları, yani testin hangi aşamasını kimin yapacağı,
- test ortamı ve kullanılacak test araçları,
- testi gerçekleştirmek için elemanların eğitim ihtiyacı,
- test takvimi,
- test risk yönetimi,
- testin beklenen çıktıları.

  Hocanın vurgusu: Yazılım geliştiriciler birim testini yapmakla yükümlüdür. Diğer test adımları ise bir test grubu tarafından yapılmalıdır.

Test takvimi için, proje planında olduğu gibi bir Gantt diyagramı hazırlanır. Hangi testin hangi zamanda ve hangisi bittikten sonra yapılacağı bu takvimde verilir. Risk yönetimi de aynı mantıkla eklenir: Yazılım proje yönetiminde nasıl bir risk yönetimi yapılıyorsa, test planı çıkarılırken de test için risk yönetimi yapılır.

9.2 Test Senaryosunun İçeriği

Bir test senaryosunda şunlar belirtilir:

- senaryonun adı, kimliği, yazarı ve tarihi,
- ilgili gereksinimler, yani senaryonun hangi gereksinim için yapıldığı,
- testin amacı,
- ön koşullar ve varsayımlar,
- test girdileri,
- senaryo adımları,
- beklenen sonuçlar.

Hocanın senaryo örneği bir öğrenci işleri sistemidir: Bir öğrencinin dönem içinde derse kaydolması bir senaryodur, bir öğrencinin transkript istemesi başka bir senaryodur.

10. Test Ekibi ve Optimum Test Miktarı

10.1 Test Ekibi

Test ekibinde bağımsızlık önemlidir. Yazılımın testini yapan grubun, yazılım geliştiriciler grubunun içinde olmaması tercih edilir. Test ekibinin görevleri şunlardır:

- planlarda belirtilen testler için gerekli hazırlıkları yapmak,
- testleri yürütmek,
- testlerde bulunan hataları yazılımcıya bildirmek,
- test raporlarını oluşturup konfigürasyon yönetimi belgelerine eklemek.

10.2 Test Ne Zamana Kadar Sürer?

Testin ne zamana kadar devam edeceği sorusuna da cevap vermek gerekir.

> **Hocanın vurgusu:** İyi bir testçi mükemmele ulaşamayacağını, iyinin yeterli olacağını bilmelidir.

Her yazılım projesinde optimum test miktarı farklıdır. Geçen hafta görülen şekilde iki eğri vardır. Bir yanda kaçırılan hata (bug) sayısını olabildiğince aşağı indirmek isteriz. Öte yanda test maliyetini çok yukarı çıkarmak istemeyiz. İki eğrinin kesişim noktası optimum test miktarıdır. Bu nokta projeden projeye değişir. Teste ayrılacak maliyetin proje maliyet planında belli bir yeri vardır. Amaç, bu maliyeti çok aşmadan kaçırılan hata sayısını olabildiğince azaltmaktır.

11. Test Faaliyetlerinin Geliştirme Aşamalarına Dağılımı

Test yalnız kodlamadan sonra yapılan bir iş değildir; planlaması geliştirme sürecinin başından başlar:

Geliştirme aşaması	Test faaliyeti
Planlama	Sistem test planlarının oluşturulması, yani kaba bir test planı
Gereksinim analizi	Alt sistemlerde her işlev belirtildikçe o işlevin nasıl test edileceğinin planlanması
Tasarım	Modül test planının oluşturulması; yapı diyagramları ve yazılım mimarisi bu plana yardım eder; test tanımları ve test eğitimi için çalışmalar
Gerçekleme	Birim testleri, bütünlük testleri ve yalnız bu parçalar için sistem testi
Kurulum	Kullanıcı testleri ve bütün üzerinde bazı sistem testleri
Teslim sonrası (bakım)	Kalitenin sürdürülmesi

12. Bakım Aşaması ve Yazılım Konfigürasyonu

Bakım, yazılımın korunması, onarılması ve geliştirilmesidir. Teslim sonrasında kalitenin sürdürülmesini sağlar. Bakımda yapılan işler şunlardır:

- sonradan görülen hataların düzeltilmesi,
- yazılımın iyileştirilmesi,
- uyarlanması,
- geliştirilmesi.

Yazılım bakımı kapsamında en çok uyarlama yapılır. Yapılan bir çalışmaya göre iyileştirme ve hata düzeltme, uyarlamanın yaklaşık yarısı kadardır.

Geniş bir programın kullanımı sırasında test aşamasında bulunmamış hatalar ve bazı tasarım hataları ortaya çıkar. Ayrıca yazılımın yeteneğini iyileştirmek ve kullanımını kolaylaştırmak için yeni yaklaşımlar ya da programlama dilindeki yeni olanaklar öğrenildikçe değişiklikler yapılabilir. Günün koşullarına göre de değişiklik gerekebilir. Yazılımın günlük hayata uyarlanmasıyla ilgili bakım işlemleri en sık yapılan işlerdir.

Bakım ve onarım giderlerini en aza indirmek için yazılım baştan bakım yapılabilir biçimde oluşturulmalıdır:

- standart programlama dilleri kullanılmalı ve belgeler standart biçimde düzenlenmeli,
- test programlarından yararlanılmalı,
- tasarım aşamasında gözden geçirme iyi yapılmalı.

Bakım yapılabilirlik (maintainability), bakım ve onarıma elverişliliği anlatır. Bunu oluşturan özellikler kolay test edilebilirlik (sınama kolaylığı), basitlik, değiştirilebilirlik, taşınabilirlik, güvenirlik ve esnekliktir.

Yazılım konfigürasyon sistemi, geliştirme sırasında her baseline'da eklenen özelliklerin bir yerde tutulmasıdır. Hoca bunu bir veri tabanı uygulaması gibi düşünmeyi önerdi. Sürekli güncellenen yazılımın yeni hâlinin bütün elemanlarını saklar ve gerektiğinde kullanıma hazır tutar.

13. Genel Tekrar: Yazılım Mühendisliğinde Kalite ve McCall Modeli

Hoca dönem başından beri görülen konuların tekrarına yeni bir slayt setiyle geçti.

13.1 Kalite Yaklaşımı ve Yazılım Mühendisliği

Yazılım mühendisliği, kalite bakış açısıyla oluşturulmuş katmanlı bir bilim dalıdır. En altta kalite bakış açısı vardır. Üstünde yazılım geliştirme süreçleri, yani seçilecek ve uygulanacak süreç adımları gelir. Onların üstünde kullanılan yöntemler, en üstte de işi kolaylaştıran yazılım araçları bulunur. Bir yazılım aracı (tool) hem kullanılan yöntemlere hem de içinde bulunulan süreç adımına uygun olmalıdır.

Bu katmanlı yapıya göre yazılım mühendisliği, kaliteli yazılımı en ekonomik şekilde ve mühendislik yöntemleriyle elde etmeye yarayan bilim dalıdır. Tanımın başında "kaliteli yazılım" geçtiği için önce kaliteli yazılımın ne olduğunu ifade etmek gerekir.

13.2 McCall Kalite Üçgeni

Kaliteli yazılımı tanımlarken McCall'un kalite üçgeni karşımıza çıkar:

Bakış açısı	Kalite faktörleri
Ürün işletimi (kullanıcı bakış açısı)	Doğruluk, güvenirlik, kullanışlılık, bütünlük, etkinlik
Ürün değişikliği	Bakım yapılabilirlik, esneklik, test edilebilirlik
Ürün geçişi	Taşınabilirlik, yeniden kullanılabilirlik, birlikte çalışabilirlik

Hoca ürün işletimi grubunda etkinliğin performansı ve dayanıklılığı, güvenirliğin de doğruluk seviyesinin güvenilir olmasını gösterdiğini hatırlattı.

13.3 Kalite Özelliklerinin Tanımları

- **Doğruluk (correctness):** Spesifikasyonlara uygunluk. Hoca bunu projeyle bağladı: Öğrencilere önce gereksinim spesifikasyonu yazdırılmasının nedeni budur. Seçilen ürünün bu gereksinim spesifikasyonuna uyup uymadığı kontrol edilecektir.
- **Güvenirlik (reliability):** Tasarlanan işlevleri istenen duyarlılıkla yerine getirme; doğruluğun ne kadar duyarlılıkla sağlandığı.

- **Verimlilik (efficiency):** Programın işlevlerini yerine getirmesi için gereken bilgi işlem kaynakları (bellek miktarı, işlem süresi) ve kodlama gideri açısından verimli olması.
- **Güvenlik / bütünlük (integrity):** Yetkisiz kişilerin yazılıma erişip erişemediği.
- **Kullanışlılık (usability):** Yazılımı öğrenmenin, işletmenin, girdi hazırlamanın ve çıktıyı yorumlamanın ne kadar kolay olduğu.
- **Bakım kolaylığı:** Hatanın yerini bulma ve düzeltme kolaylığı.
- **Test edilebilirlik:** Yazılımın doğruluğunu test etmenin ne kadar kolay olduğu.
- **Taşınabilirlik:** Yazılımın farklı donanım ve işletim yazılımı üzerinde kullanılabilmesi. Bir yazılım başka bir donanım ya da yazılım ortamına düşük maliyetle taşınabiliyorsa taşınabilirdir.
- **Yeniden kullanılabilirlik (reusability):** Yazılımın tümünün veya bir bölümünün başka bir uygulamada kullanılabilmesi. İş gücünü verimli kullanmak için bu geliştirme mantığı önemlidir.
- **Birlikte çalışabilirlik (interoperability):** Bir sistemin diğer bir sistemle birlikte çalışabilmesi.

13.4 Kalite Faktörleri ile Metrikler Arasındaki İlişki

Kalite modelinde sol tarafta kalite faktörleri, sağ tarafta metrikler bulunur. Metriklerin sayısı çoktur; derste bir kısmı görülmüştü. Bazen bir kalite özelliği, metrik tarafında başka bir kalite özelliğini bulmaya hizmet eder. Örnekler:

- Birlikte çalışabilirlik için modülerlik, iletişim genelliği ve veri genelliği önemlidir.
- Yeniden kullanılabilirlik için yazılım sistemi bağımsızlığı ve makine bağımsızlığı önemlidir.
- Taşınabilirlik ve basitlik de başka bir faktör için metrik olarak geçer.

  Hocanın vurgusu: Her bir kalite faktörü doğrudan ölçülmez. Metrikler bir takım katsayılarla bir regresyon denkleminin içinde kullanılarak hesaplanır.

14. Kaliteli Yazılım İçin Aktiviteler ve Kalite Maliyeti

Kaliteli yazılımı tanımladıktan sonra onu elde etmek için yapılması gerekenlere geçildi:

- standartlara uymak,
- doğrulama ve geçerleme (verification, validation),
- gözden geçirme ve denetim (review),
- test,
- hata ve kusur analizi,
- değişim yönetimi ve konfigürasyon yönetimi,
- eğitim çalışmaları,
- risk yönetimi: yazılımı geliştirirken ve test ederken her aşamada önerilir.

Bu aktivitelerin teslimden önce yapılması gerekir. Hata yazılım evinden çıkmadan, ya da başka bir yazılımla etkileşimdeki sorun ortaya çıkmadan bulunup düzeltilmelidir.

Hoca maliyet tahmini yaparken şu ayrımın akılda tutulmasını istedi:

- **Hata önleme maliyeti:** Kaliteli yazılım elde etmek için harcanır. Kalite planlama, formal teknik gözden geçirme, test oluşturma ve eğitim bu maliyete girer.
- **İç hata maliyeti:** Yazılım daha yazılım evinin içindeyken, kullanıcıya verilmeden bulunan hataların maliyeti.
- **Dış hata maliyeti:** Yazılım teslim edilip kullanıma alındıktan sonra ortaya çıkan hataların maliyeti. Bu maliyet daha yüksektir ve başka kötülükler de getirir: Şirketin tanınırlığına ve itibarına zarar verir.

15. Standartlar ve Kalite Yönetim Sistemi

Kalite aktivitelerinden ilki standartlardır. Standart bir kurum standardı, bir proje standardı ya da yazılım konusunda dünya çapında kabul görmüş bir standart olabilir.

Hoca slayttaki şemayı şöyle anlattı: Bir yanda dünya çapında kabul gören genel standartlar, öbür yanda bir projenin standart el kitabı vardır. Şirkette geliştirilen bir projeye başlandığında

kurumun zaten bir kalite yönetim sistemi ve buna bağlı bir kalite el kitabı vardır. Kalite kontrolünde kullanılacak standartlar ve diğer yardımcı hizmetler de buradadır. Her proje için kurum standartlarına ekler yapılarak proje standartları genişletilebilir.

Kalite yönetim sistemi, geliştirilecek yazılımın istenen kalite faktörlerine sahip olmasını sağlayan, geliştirici ve kullanıcının birlikte belirlediği bir yönetim yapısıdır. Hangi sorumlulukların bulunduğunu ve hangi kaynaklardan oluştuğunu belirler.

16. V Model ve Doğrulama-Geçerleme Düzeltmesi

Doğrulama ve geçerleme her aşamada yapılır ve V model üzerinde çok durulmuştu. Hoca bu tekrar slaytının eskiden kalma olduğunu, doğrulama ile geçerlemenin yerlerinin yanlış yazıldığını söyledi ve slaytta düzeltti.

> **Hocanın vurgusu:** Aklınızda yanlış kalmasın. Validation geçerleme, verification doğrulama olarak kabul edildi.

V modelde yazılım geliştirme süreci ile yazılım test süreci ilişkilendirilir. Sol taraf yazılım geliştirme sürecidir, sağ taraf yazılım test sürecidir. Test süreci boyunca hangi düzeyde değerlendirme yapıldığı da gösterilir. Bileşen testi ile alçak ve yüksek düzey tasarım düzeylerinde genellikle doğrulama (verification) yapılır. Kullanıcı testinde ise geçerleme (validation) yapılır; yani ürünün gereksinimlere uygun olup olmadığına bakılır.

17. Gözden Geçirme ve İnceleme

V modele ve teste gelmeden önce gözden geçirme ve inceleme işlenmişti.

Gözden geçirme, yazılım geliştirme sürecini oluşturan her basamak tamamlandığında durak noktalarında yapılan işlerin gözden geçirilmesidir. İlk hedef hataları bulmak ve giderilmesini istemektir. Ayrıca yazılımın standartlara uygun yazılıp yazılmadığı da kontrol edilir. Yalnız doğru cevap veriyor olması yeterli değildir.

Gözden geçirme ve inceleme tipleri şunlardır:

- formal olmayan gözden geçirme (informal review),
- yapısal denetim (walkthrough),
- inceleme (inspection).

Bunlar her aşamada uygulanır. Sistem analizinde, yapısal geliştirme planında, gereksinim analizi ve tasarım aşamalarında durak noktalarının (milestone) tanımlanması gerekir.

İnceleme (inspection) aşamasında hedef, kodları çalıştırmadan hataları bulmaktır. Bu bir statik geçerleme tipidir. Hoca önceki derslerde gözden geçirme ile inceleme arasındaki farklar üzerinde durulduğunu hatırlattı.

18. Test Süreci Adımları

Üçüncü aradan sonra hoca tekrarın akışını özetledi: İlk üç derste yazılım mühendisliğinde kalitenin yeri, kalite özellikleri ve kaliteyi sağlamak için gereken aktiviteler görülmüştü. Bu aktivitelerden geliştirmenin başında standardın belirlenmesi, her adımda gözden geçirme ve inceleme yapılması işlenmişti. Ardından test aşamasına geçilmişti. Testler yazılımı performans, fonksiyonellik, dayanıklılık ve yapı açısından inceler.

V modelin sağ tarafındaki adımlar, yani yazılım test stratejisi sırasıyla şöyledir.

18.1 Birim Testi

Birim testinde test senaryosunun (test case) ne olduğu, test programlarının faydası, stub ve driver kavramları işlenmişti.

18.2 Bütünlük Testi

Bütünlük testi (integration test), yazılım tasarımı sırasında, özellikle genel tasarımda oluşturulan mimari yapı üzerinde yapılır. İki yaklaşım vardır:

- **Aşağıdan yukarıya (bottom-up):** En alttaki en küçük modüllerden başlanır, önce bunların testi tamamlanır. Sonra modüller birleştirilerek, bir anlamda kümeler (cluster) oluşturularak yukarı doğru ilerlenir ve en sonunda bütün test edilir.
- **Yukarıdan aşağıya (top-down):** Önce en üstteki genel yapı test edilir. Alt modüllerin yerine yapay modüller (stub) konur; birim testinde çağrılan fonksiyonların yerine stub konmasına benzer. Sonra ayrıntılar aşağı doğru açılarak test edilir.

18.3 Regresyon Testi

Regresyon testi, test edilmiş bir program ya da program parçası üzerinde değişiklik veya ekleme yapıldığında programın bir kez daha test edilmesidir. Gerekli değişiklikler yapıldıktan sonra alt modüllerde ya da uygulama ortamlarında yeniden yapılan testlere regresyon testi ya da tamamlama testi denir.

18.4 Sistem Testi

Yazılım tek başına değil, alt yazılımlarla ve birlikte çalıştığı ortamla birlikte çalışır. Sistem testi, sistemin bütün öğelerinin birbiriyle uyumlu çalıştığının onaylanmasıdır. Şu biçimlerde yapılır:

- **Güvenlik testi:** Sistem dış müdahalelerden ve bilgi hırsızlığından korunuyor mu?
- **Dayanıklılık (stres) testi:** Sistem fazla yüklendiğinde bundan başarıyla çıkabiliyor mu, başarı oranı nedir?
- **Yetenek (performans) testi:** İşlem süresi ve yetenek açısından değerlendirme.

Sistem testinin "her test basamağında uygulanabilmesi" şu anlama gelir: Normalde sistem testi bütünlük testinden sonra, bütün doğru çalışıyorsa sistemin de doğru çalışıp çalışmadığını görmek için yapılır. Ama bir modül içinde de uygulanabilir. Bir modül fonksiyonlarını yerine getirip doğru sonuç verse bile işlem süresi ve dayanıklılık açısından o modüle de sistem testi uygulanabilir.

18.5 Onaylama Testi

Onaylama testinde, bütünlük testi sonunda hataları ayıklanmış yazılımın teslim edilip edilemeyeceği kullanıcı tarafından onaylanır. İki parçası vardır:

- **Kabul muayenesi:** Sistem doğru çalışıyorsa, bu doğru çalışma kullanıcı belgeleri izlenerek sağlanabiliyorsa ve sistemle ilgili bütün belgeler tamamsa yazılım kabul edilir.
- **Düzen testi:** Burada konfigürasyon "düzen" anlamında kullanılır. Yazılımın konfigürasyonu, yani bütün sistem belgeleri, kullanıcı belgeleri, test verisi ve test spesifikasyonu eksiksiz verilmiş mi diye bakılır.

Kabul muayenesinden ve düzen testinden geçen sistem için onaylama gerçekleşmiş olur. Bu, hem sistemin tüm elemanlarıyla çalıştığının hem de bütün konfigürasyon elemanlarının bulunduğunun onaylanmasıdır; sistemin teslimine onay verilir.

19. Test Tipleri: Saydam Kutu, Kara Kutu, Gri Kutu

Test süreci adımları testin hangi sırayla yapıldığını, test tipleri ise testin ne şekilde yapıldığını belirler. İki temel test tipi kara kutu ve saydam kutu testidir. İkisinin karması olarak yürütülen gri kutu (gray box) testi de vardır.

Test süreci adımlarının her birinde hem kara kutu hem saydam kutu testi yürütülebilir. Ancak birim testinde ağırlıklı olarak saydam kutu, kullanıcı testinde ağırlıklı olarak kara kutu testi yapılır. Ayrımın ölçüsü şudur: Test, yazılımın dış görünüşü ve dış tepkileri açısından mı yapılıyor, yoksa iç ayrıntısı, algoritması ve algoritma yapıları açısından mı? İkincisinde yapısal denetim yapılır ve buna saydam kutu testi denir.

19.1 Saydam Kutu Testi

Saydam kutu (white box) testinde ayrıntılı tasarımda oluşturulan algoritma kullanılır. Bu testte şunlar sağlanmalıdır:

- Bir modüldeki bütün bağımsız yollar en az bir kez çalıştırılmış olmalıdır.
- Bütün mantıksal kararlar için doğru ve yanlış durumları denenmiş olmalıdır.
- Bütün döngüler kendi içinde artırma değerleri ve işlerlikleri açısından değerlendirilmiş olmalıdır.
- İç veri yapıları test edilmiş olmalıdır.

Saydam kutu test tipinin örnekleri, kontrol deyimleri üzerinden yapılan temel yol testi ve döngü testidir.

Temel yol testi (basis path testing): Saydam kutu testlerinin en önemlisidir. Algoritmadaki bütün temel yollar belirlenir ve her biri için test verisi hazırlanabilmesi sağlanır. Her yoldan en az bir kez geçilmeden test aşaması tamamlanmamalıdır. Adımlar:

- Akış grafı çizilir.
- Bu graf üzerinde, ölçütler konusunda görülen McCabe ölçüsü (döngüsel karmaşıklık) bulunur.
- Doğrusal bağımsız yolların temel grubu belirlenir.
- Her temel yoldan bir kez geçmek için gereken test verisi hazırlanır ya da test senaryoları oluşturulur.
- Test programı çalıştırılır ve çıkan sonucun beklenen sonuçla aynı olup olmadığı kontrol edilir.

Döngü testi: Bütün döngüler çalıştırılır. Döngülerin (loop, repeat-until ve while blokları) kontrol deyimleri ve artışları kontrol edilir. Sonsuz döngü olup olmadığına ve döngüye hiç girilip girilmediğine bakılır.

19.2 Kara Kutu Testi

Kara kutu (black box) testi, yazılıma dışarıdan, onu çalıştıran kişinin bakış açısıyla bakılan testtir. Yazılımın işlevlerini yerine getirip getirmediği ve girdileri kabul edip etmediği kontrol edilir. Yazılımın mantıksal iç yapısından çok temel sistem yapısı test edilmiş olur.

> **Hocanın vurgusu:** Kara kutu ve saydam kutu testleri birbirinin tamamlayıcısıdır. "Önce biri, sonra öbürü" diye bir sıra yoktur. Birim testinde ağırlıklı olarak saydam kutu, kullanıcı testinde ağırlıklı olarak kara kutu kullanılsa da ikisi birlikte kullanılarak test sürdürülür.

Kara kutu testinde iç yapı görülmediği için, kullanıcı olarak bütün yollardan geçilip geçilmediği ayırt edilemez. Bu yüzden daha az testle farklı durumları deneyebilmek için veri sınıfları oluşturulur ya da sınırlara bakılır:

- **Eş değerli bölümleme (equivalence partitioning):** Belli sınıflar, yani değer aralıkları oluşturulur ve yazılım bu aralıklardan seçilen değerlerle çalıştırılır. Böylece farklı veri grupları üzerindeki tepkileri denenmiş olur.
- **Sınır değer analizi (boundary value analysis):** "Hatalar sınırlarda olur" varsayımıyla öncelik sınırların denenmesine verilir. Test sayısı elverdikçe daha sonra farklı değerler verilerek iç değerler de kontrol edilebilir.

20. Hata Giderme ve Hata Düzeyleri

Hata giderme (debugging), test sonucunda saptanan hataların nedenlerini bulup düzeltme sürecidir. Diğer derslerde hata nedenleri üzerinde durulmuştu. Sıralamayla gidilirse hatalar gereksinimdeki hata ve eksikliklerden, tasarımdan, koddan ve diğer aşamalardan kaynaklanır.

Kalite yaklaşımını uygulayanlar düzeyinde hata düzeyleri de tanımlanmalıdır: ölümcül, kritik, büyük, orta, küçük ve görünüm hataları. Hata düzeyinin test ve teslim üzerindeki etkisi şöyledir:

Hata düzeyi	Test	Teslim
Ölümcül	Durdurulur; hata düzelmeden teste devam edilmez	Durdurulur

Hata düzeyi	Test	Teslim
Kritik	Bazı kalite gruplarına göre durdurulur, bazılarına göre devam edebilir	Durdurulur
Büyük	Durdurulmaz	Derste açıkça belirtilmedi
Orta	Durdurulmaz	Bazen orta hatalarla da teslime gidilir
Küçük ve görünüm	Durdurulmaz	Teslim aşamasına geçilebilir

> **Hocanın vurgusu:** Ölümcül bir hata varsa yalnız teslim durdurulmaz, test de durdurulur.

Hoca tekrarı, kalite ölçüm ve değerlendirme konusunun bu aşamalardan sonra geldiğini ve bugün o konunun da ele alındığını söyleyerek bitirdi. Tekrar slaytlarında bazı ayrıntıların bulunmadığını, ancak bunların ders anlatımlarında geçtiğini hatırlattı. Test çalışma ekibi, test planlama ve yönetimi ile test senaryolarının oluşturulması da tekrar edilen konular arasındaydı.

Örnek Sorular

Bu derste kare yoktur. Aşağıdaki sorular hocanın derste sözlü olarak sorduğu ve kendisinin cevapladığı sorulardır.

Soru 1 (derste sözlü, 00:15:48): Yazılımın güvenirliği hakkında ne söyleyebiliriz, güvenirlik nasıl ölçülür?

> **Çözüm:** Güvenirlik bir kalite özelliğidir. Belli bir kullanım süresinde kaç hata çıktığı, bunlardan kaçının düzeltilebildiği ve düzeltmeden sonra ne kadar hata kaldığı gibi istatistiksel sayımlarla güvenirlik modellerine ulaşılır. Böylece sistemin hangi güvenirlik oranıyla çalıştığı söylenebilir.

Soru 2 (derste sözlü, 00:22:44): Function point hesabında neleri sayıyorduk? Function point doğrudan mı, dolaylı mı bir ölçümdür?

> **Çözüm:** Girdi sayısı, çıktı sayısı ve dosya sayısı sayılır. Bunlar doğrudan ölçümdür. Function point bu sayılardan hesaplandığı için dolaylı bir ölçümdür.

Soru 3 (derste sözlü, 00:24:36): Yazılımı neden ölçüyoruz?

> **Çözüm:** Ne derecede iyi bir ürün ortaya çıkardığımızı bulmak, bu büyüklükte bir yazılım için ne kadar zaman ve para harcadığımızı bilmek ve yeni bir yazılımda büyüklük tahmini yapıp maliyet kestirmek için. Ayrıca kaliteyi artırmak için neyin değişmesi gerektiğini bilmek, var olan durumu ölçmeyi gerektirir: “Ölçülemeyen yönetilemez.”

Soru 4 (derste sözlü, 00:27:21): Dışarıdan bakan kişi, yani kullanıcı, yazılımdan ne ister? Hangi özellikler üzerinde durur?

> **Çözüm:** Kullanışlılık, performans (hızlı cevap süresi, veri kapasitesi), yetkisiz kullanımı önleyen güvenlik, doğru çalışma ve bu doğruluğun güvenilir olması (reliability). Bunlar McConnell’ın dış kalite ölçütleridir.

Soru 5 (derste sözlü, 01:15:58): Test ne zamana kadar devam edecek?

> **Çözüm:** İyi bir testçi mükemmele ulaşamayacağını, iyinin yeterli olacağını bilmelidir. Optimum test miktarı, kaçırılan hata sayısı eğrisi ile test maliyeti eğrisinin kesiştiği noktadır ve her projede farklıdır.

Soru 6 (derste sözlü, 01:31:17): Yazılım kalite özelliklerinden yeniden kullanılabilirlik (reusability) neydi?

> **Çözüm:** Yazılımın tümünün veya bir bölümünün başka bir uygulamada kullanılabilmesidir. İş gücünü verimli kullanmak için önemlidir.

Soru 7 (derste sözlü, 01:43:12): Gözden geçirme nedir?

Çözüm: Yazılım geliştirme sürecinin her basamağı tamamlandığında durak noktalarında yapılan işlerin gözden geçirilmesidir. İlk hedef hataları bulup giderilmesini istemektir. Standartlara uygunluk da kontrol edilir, çünkü yalnız doğru cevap vermek yeterli değildir.

Ders sonrası notlar

Ders sonrası not: Altyazıda “Maccape” ve “makyayip” McCabe, “Halçtaht” ve “hashtag metriği” Halstead, “Macconnell” ve “Macanl” McConnell, “Maccon’un kalite üçgeni” McCall’un kalite üçgeni, “saklomatik kompleksite” cyclomatic complexity, “Likom” LCOM, “programın harcımı” programın hacmi, “Ecolons Partitioning” equivalence partitioning, “Bonderweilu” boundary value, “bezlen” baseline, “bilim test” birim test, “stop” stub olarak okunmalıdır.

Ders sonrası not: Hoca McCall’un gruplarını “işlevsel, değiştirilebilme, çalıştırılabilme ölçütleri” diye andı. Literatürdeki adlar ürün işletimi (product operation), ürün değişikliği (product revision) ve ürün geçişidir (product transition); dersin tekrar slaytında da bu üçlü kullanıldı.

Ders sonrası not: Altyazıda “kaçılan box sayısı” geçiyor; kastedilen kaçırılan hata (bug) sayısıdır.

Ders sonrası not: Hoca bütünlük testinde stub kavramını yukarıdan aşağıya yaklaşımla açıkladı. Aşağıdan yukarıya yaklaşımda ise alt modülleri çağıracak üst modülün yerini sürücü (driver) alır.

Dersin Toparlaması

Bu dersin ilk yarısında yazılım ölçümü yazılım mühendisliği bakış açısıyla tekrar edildi. Halstead, McCabe, büyüklük ve güvenirlik ölçüleri hatırlatıldı. Ölçme, ölçüt ve ölçüm kavramları ayrıldı; function point örneğiyle doğrudan ve dolaylı ölçüm farkı gösterildi. McConnell’ın iç ve dış kalite ölçütleri, beş adımlı ölçme süreci (tanımlama, toplama, hesaplama, yorumlama, geri besleme) ve nesneye yönelik yazılımlar için CK metrikleri işlendi. Ardından test planlama ve yönetimi tekrar edildi: test spesifikasyonu, test planı ve test senaryosu içerikleri, test ekibinin bağımsızlığı, optimum test miktarı, test faaliyetlerinin geliştirme aşamalarına dağılımı ve bakım.

Dersin ikinci yarısı dönem başından beri görülen konuların genel tekrarıydı: yazılım mühendisliğinde kalitenin yeri, McCall kalite üçgeni ve kalite özellikleri, kalite aktiviteleri ve kalite maliyeti, standartlar ve kalite yönetim sistemi, V model (doğrulama ve geçerleme düzeltmesiyle), gözden geçirme ve inceleme, test süreci adımları, test tipleri, hata giderme ve hata düzeyleri. Hoca, projelerde test aşamasının nasıl yürütüleceğini ve kalite iyileştirme konusunu sınavdan sonraki haftalarda ele alacağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.